VHL Ecosystem Architecture

Virtual Harmonium Lab — VHL Specification v1.0

Engineering Architecture Document

Status: Frozen — VHL v1.0 Specification
Date: 2026-07-29
Applies to: VHL Builder, ESP32 Runtime, all future instruments

---

Table of Contents

1. Design Philosophy
2. Binary Layout
3. Builder Architecture
4. Runtime Architecture
5. Memory Architecture
6. Builder vs Runtime Responsibilities
7. Future Expansion

---

1. Design Philosophy

Why VHL Exists

Professional sampler formats — SF2 and SFZ — were designed for desktop computers. They assume large RAM, fast CPUs, operating systems, filesystems, dynamic memory allocation, and complex parsers. An ESP32 has none of these. It has 320 KB of internal SRAM, no OS, no heap suitable for audio, and a 240 MHz core that must complete one audio buffer every 11.6 milliseconds without fail.

Attempting to run an SF2 parser on an ESP32 is not merely difficult — it is architecturally incorrect. SF2 binary parsing, SFZ text parsing, preset-to-instrument hierarchy traversal, modulator graph resolution, and runtime parameter merging are all operations that belong on a desktop computer. They are preprocessing steps, not real-time steps.

VHL (Virtual Harmonium Lab format) exists to separate these two concerns completely. It is a binary format designed from scratch for a single purpose: to be read by an embedded microcontroller with minimal CPU and memory usage, enabling immediate audio playback with no interpretation overhead.

Why Not Use SF2 Directly

SF2 on the ESP32 would require:

- A binary chunk parser (RIFF structure traversal)
- Dynamic memory allocation for parsed data
- Two-stage hierarchy traversal (preset → instrument → sample) at every Note-On
- Modulator graph resolution at every Note-On
- Runtime parameter merging (global zone + local zone for every generator)
- `pow()`, `log()`, and `exp()` calls for parameter conversion at Note-On

Each of these takes between 10 and 500 microseconds. A Note-On must be processed in under 20 microseconds to be imperceptible. SF2 on ESP32 would produce an instrument that feels sluggish and unresponsive — unacceptable for a musical instrument.

Additionally, SF2 is a proprietary binary format designed for files, not for microcontroller memory maps. It contains redundant data, human-readable strings, editor metadata, and compression artifacts that are irrelevant to a playback engine.

Why Convert from SFZ (Not SF2)

Polyphone is the instrument design tool. Polyphone exports SFZ — a text-based, human-readable format that describes an instrument as a collection of regions. SFZ is simpler than SF2 to parse in the Builder and more transparent to debug.

SFZ provides everything needed for the VHL conversion pipeline:

- Region definitions with key and velocity ranges
- Envelope parameters
- Filter parameters
- LFO parameters
- Loop configuration
- Sample file references

Importantly, Polyphone's SFZ export already resolves some SF2 complexity (the preset/instrument distinction collapses into flat regions in SFZ). This simplifies the Builder's inheritance resolution stage.

Why the Builder Performs Preprocessing

Every computation that can be performed once, offline, on a powerful desktop computer must be performed there — not on the ESP32.

The Builder has unlimited time. It can spend 30 seconds computing lookup tables, resolving inheritance, running `pow()` thousands of times, and validating every parameter. The ESP32 has 11.6 milliseconds. These two time budgets are incompatible with sharing computational work.

The Builder's output is a VHL file that contains only the data the runtime actually needs, in the exact form it needs it, with every expensive computation already complete. The runtime reads binary structures and synthesizes audio. It does not interpret, parse, or compute anything that could have been done offline.

This is the foundational principle: **the Builder is arbitrarily complex so that the Runtime can be arbitrarily simple.**

Why the Runtime Performs Only Playback

An embedded real-time audio engine has one job: fill an audio buffer before the hardware empties it. Every microsecond spent on parsing, interpretation, or offline computation is a microsecond stolen from the audio engine's time budget.

The VHL Runtime is a pure playback engine. It knows nothing about SFZ syntax, SF2 structure, modulator graphs, or inheritance hierarchies. It knows about voices, envelopes, filters, and samples. It reads pre-baked binary data and produces audio. This simplicity is what makes it fast, correct, and maintainable.

---

2. Binary Layout

Every VHL file contains exactly six sections in the following order. Sections are referenced by absolute byte offsets stored in the File Header, so the runtime can jump to any section immediately without sequential scanning.

```
┌──────────────────────────────────────────────────────────────┐
│ SECTION 0: FILE HEADER                          128 bytes    │
├──────────────────────────────────────────────────────────────┤
│ SECTION 1: INSTRUMENT TABLE                                  │
│   num_instruments × 40 bytes                                 │
├──────────────────────────────────────────────────────────────┤
│ SECTION 2: REGION TABLE                                      │
│   total_region_count × 80 bytes                              │
├──────────────────────────────────────────────────────────────┤
│ SECTION 3: SAMPLE DIRECTORY                                  │
```

```
|   total_sample_count × 32 bytes                                          |
├──────────────────────────────────────────────────────────────────────────┤
| SECTION 4: LOOKUP TABLE                                                  |
|   num_instruments × (4 + 128×4) bytes = 516 bytes per instrument         |
├──────────────────────────────────────────────────────────────────────────┤
| SECTION 5: PCM SAMPLE DATA                                               |
|   Variable — typically 80–95% of total file size                         |
└──────────────────────────────────────────────────────────────────────────┘
```

Section 0: File Header (128 bytes, fixed)

Purpose: Self-identification and section index.

Contents:

- `magic[4]` — The bytes `{'V','H','L','1'}`. The runtime refuses to load any file whose first four bytes do not match this sequence. This prevents accidental loading of wrong files.
- `version_major`, `version_minor` — Format version. The runtime refuses files whose major version does not match its own. Minor version mismatches are tolerated (additive changes only).
- `flags` — File-level feature flags (stereo output, lookup table present, sample preload hint).
- `num_instruments`, `total_region_count`, `total_sample_count` — Global counts for sizing array reads at boot.
- `target_sample_rate`, `dma_block_size` — Audio configuration hints for the runtime I^2S setup.
- Ten `offset_*` fields — Absolute byte offsets to each section. The runtime uses these to jump directly to any section without sequential traversal.
- `total_file_size`, `crc32` — Integrity. The runtime computes the CRC32 of the entire file (with the `crc32` field set to zero) and compares with the stored value. Mismatch → refuse to load.
- `build_timestamp`, `builder_version`, `build_tag` — Metadata for debugging and version tracking.

Why it exists: A fixed-size, fixed-location header at byte 0 is the most efficient possible way to describe a binary file. The runtime can read exactly 128 bytes, validate, and then jump to any section in one operation. No scanning required.

Approximate size: 128 bytes (fixed).

Runtime usage: Read once at boot. Validated immediately. Section offsets extracted and stored in a local runtime structure.

Builder usage: Written last, after all other sections, so that all offsets and sizes are known.

Section 1: Instrument Table

Purpose: Maps MIDI bank+program numbers to instrument metadata and locates their regions.

Contents: An array of `VHLInstrumentEntry` structs (40 bytes each), one per instrument. Each entry contains:

- Instrument ID and MIDI bank+program number for MIDI routing
- Name string (24 bytes, null-terminated) for display
- `region_offset` and `region_count` — Slice into the global Region Table for this instrument
- `lookup_table_offset` — Byte offset to this instrument's lookup acceleration data

Why it exists: The instrument table allows the runtime to load one VHL file and support multiple instruments (harmonium, flute, tanpura, etc.) without reloading the file. MIDI program change selects a different instrument by consulting this table.

Approximate size: 32 instruments × 40 bytes = 1.28 KB (maximum).

Runtime usage: Loaded into SRAM at boot. Consulted on MIDI program change and at every Note-On to locate the correct region slice.

Builder usage: Generated after all regions are assigned, so that `region_offset` and `region_count` can be filled in correctly.

Section 2: Region Table

Purpose: Contains every zone definition for every instrument, completely flat and pre-baked.

Contents: An array of `VHLRegion` structs (80 bytes each). Each region contains:

- Key and velocity range (`key_lo`, `key_hi`, `vel_lo`, `vel_hi`) for zone selection
- `sample_id` — Direct index into the Sample Directory
- `exclusive_class`, `loop_mode`, `flags` — Voice behavior
- `coarse_tune`, `fine_tune`, `scale_tuning` — Tuning
- `attenuation_cb`, `pan`, `reverb_send`, `chorus_send` — Volume and spatial
- `vol_env` — Complete volume envelope (`VHLEnvelope`, 14 bytes)
- `mod_env` — Complete modulation envelope (`VHLEnvelope`, 14 bytes)
- `mod_env_to_pitch`, `mod_env_to_filter` — Modulation routing
- `lfo_mod` — Complete modulation LFO (`VHLModLFO`, 12 bytes)
- `lfo_vib` — Complete vibrato LFO (`VHLVibLFO`, 8 bytes)
- `filter` — Filter cutoff and resonance (`VHLFilter`, 4 bytes)

The flat design is the most important architectural decision in the entire format. In SF2, each zone has a pointer to a global zone, which inherits from an instrument, which inherits from a preset global zone. In VHL, inheritance has been resolved at build time. Every region is self-contained. The runtime reads one struct and knows everything.

Why it exists: The flat region design eliminates all hierarchy traversal, pointer chasing, and parameter merging from the runtime. A Note-On produces a list of matching region indices. The runtime reads each `VHLRegion` directly and begins voice initialization immediately.

Approximate size: 100 regions × 80 bytes = 8 KB (typical harmonium). Maximum 4096 × 80 = 320 KB.

Runtime usage: Loaded into SRAM at boot. Indexed directly by region_idx at Note-On.

Builder usage: The largest single output of the Builder. Produced by the hierarchy resolution and parameter baking stages.

Section 3: Sample Directory

Purpose: Metadata for every sample — location, size, loop points, root key, sample rate.

Contents: An array of `VHLSampleEntry` structs (32 bytes each). Each entry contains:

- `data_offset` — Byte offset from the start of the PCM Data section to the first frame of this sample. The runtime jumps here directly.
- `total_frames`, `loop_start_frame`, `loop_end_frame` — Absolute frame positions. All offsets and generators have been merged at build time.
- `sample_rate` — Source rate, used in the pitch ratio computation.
- `root_key` — The MIDI note at which the sample sounds its natural pitch.
- `fine_tune_cents` — Sample-level fine tuning.
- `bit_depth`, `channels`, `flags` — Format properties.
- `linked_sample_id` — For stereo pairs: the ID of the partner sample.

Why it exists: Separating sample metadata from PCM data allows the runtime to load all metadata into SRAM (tiny) while keeping PCM data in flash or PSRAM. The runtime can compute a pointer to any sample's PCM data in one addition: `pcm_base + entry.data_offset`.

Approximate size: 100 samples × 32 bytes = 3.2 KB (typical). Maximum 1024 × 32 = 32 KB.

Runtime usage: Loaded into SRAM at boot. Indexed by `VHLRegion.sample_id` at voice initialization.

Builder usage: Written after all samples are encoded and their positions in the PCM section are known.

Section 4: Lookup Table

Purpose: Accelerates region selection at Note-On from O(N) to effectively O(1).

Contents: For each instrument:

- One `VHLLookupHeader` (4 bytes) — instrument identification
- 128 `VHLLookupEntry` structs (4 bytes each) — one per MIDI key

Each `VHLLookupEntry` contains:

- `region_start_idx` — The first region index in the Region Table that could match this key (or `VHL_LOOKUP_NONE` if no regions match this key at all)
- `region_count` — How many consecutive regions to scan for this key

At Note-On: look up `entries[played_key]`. If `region_start_idx == VHL_LOOKUP_NONE`, no regions match this key at all — skip immediately. Otherwise, scan exactly `region_count` regions, checking only velocity range. For a harmonium with one region per key, this is one comparison.

Why it exists: Without the lookup table, a Note-On requires scanning all regions to find matches. With 100 regions this is fast, but with 1000 regions (a full multisample library) it becomes measurable. The lookup table eliminates key-range comparisons entirely and bounds the velocity scan to a tiny number of candidates.

Approximate size: 1 instrument × (4 + 128×4) = 516 bytes. 32 instruments × 516 = 16.5 KB maximum.

Runtime usage: Loaded into SRAM at boot. Consulted at every Note-On.

Builder usage: Generated last, after all regions have been assigned to instruments and sorted by key range.

Section 5: PCM Sample Data

Purpose: Raw audio data — the actual waveforms that produce sound.

Contents: Concatenated 16-bit signed PCM frames, one sample after another. Each sample begins at a 4-byte aligned address. The Builder inserts padding bytes between samples as needed for alignment.

There is no structure to this section — it is a flat byte array. The `VHLSampleEntry.data_offset` field provides the byte offset to each sample's data.

Why it exists: PCM data is fundamentally different from metadata — it is large (megabytes), accessed sequentially, and never needs to be modified at runtime. Placing it at the end of the file allows

everything else to be loaded into SRAM while the PCM data remains in flash or PSRAM, accessed by pointer.

Approximate size: 44 reed samples (harmonium) × ~100 KB average = ~4.4 MB. This is 95–98% of a typical VHL file.

Runtime usage: Not loaded into SRAM. Accessed by pointer arithmetic: `pcm_base + VHLSampleEntry.data_offset`. If PSRAM is available, the entire section is copied there at boot. Otherwise, accessed via XIP (Execute-In-Place) flash cache.

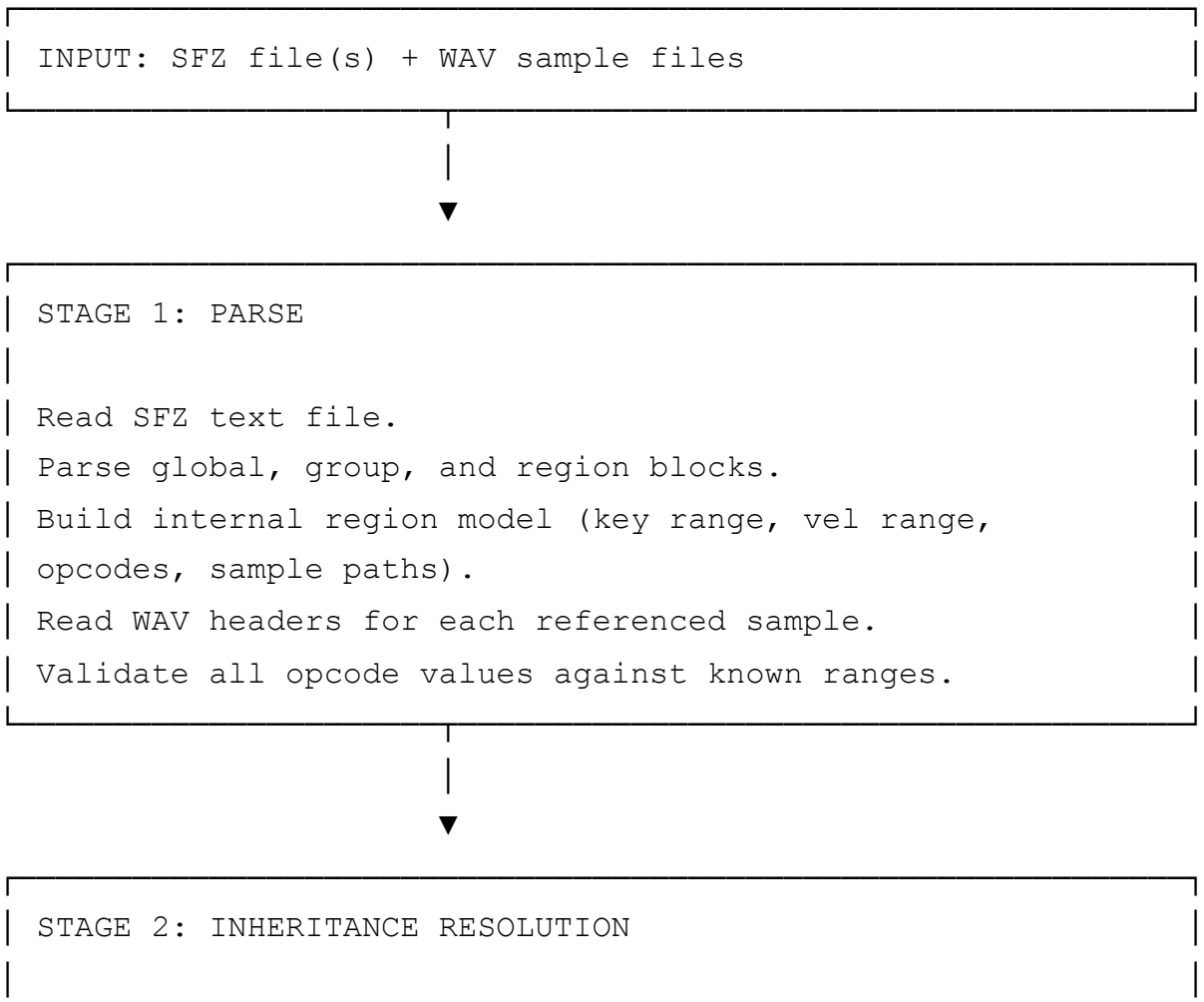
Builder usage: Written sample by sample, with padding inserted for alignment. Each sample's `data_offset` is recorded for the Sample Directory.

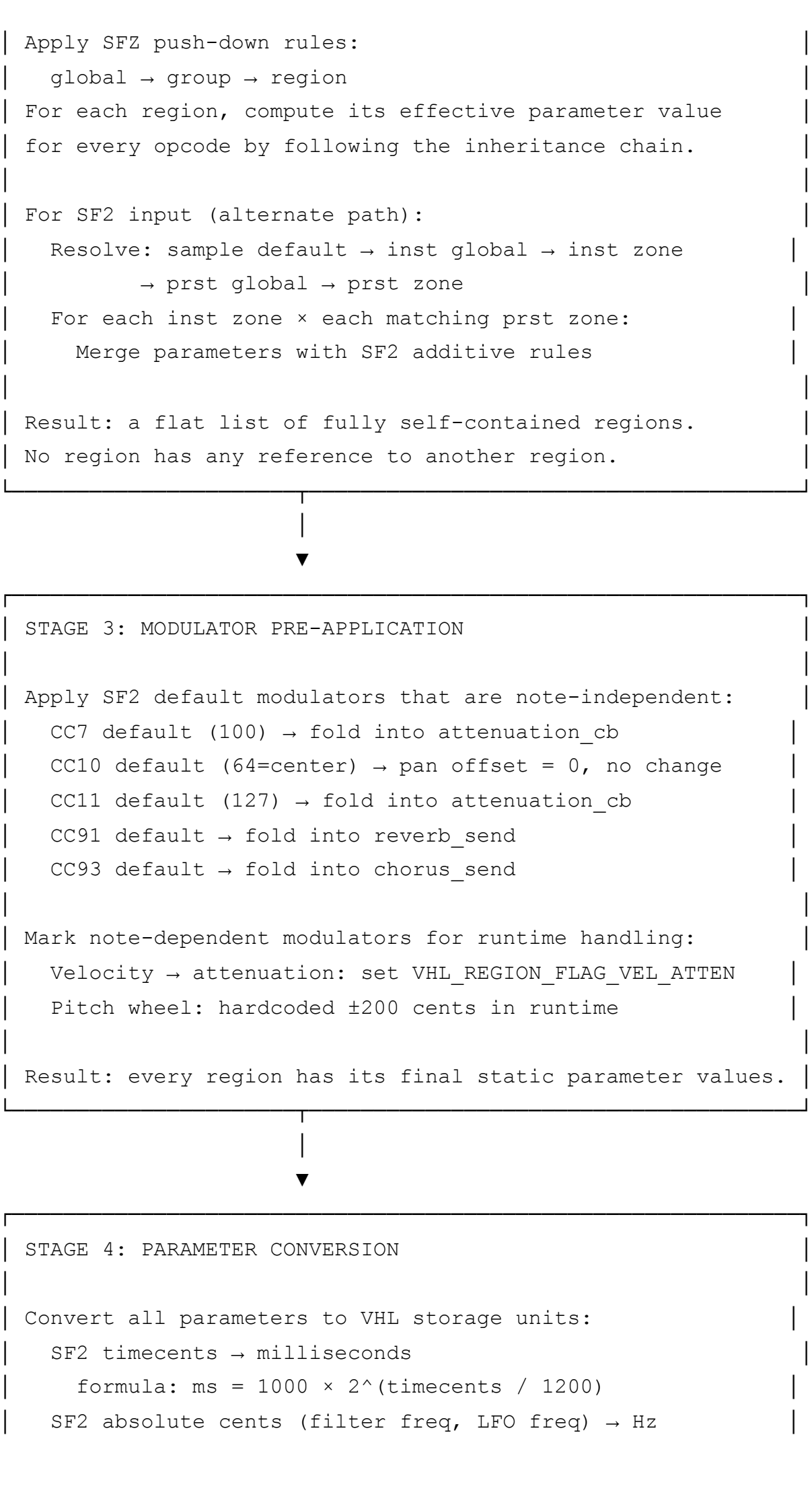
---

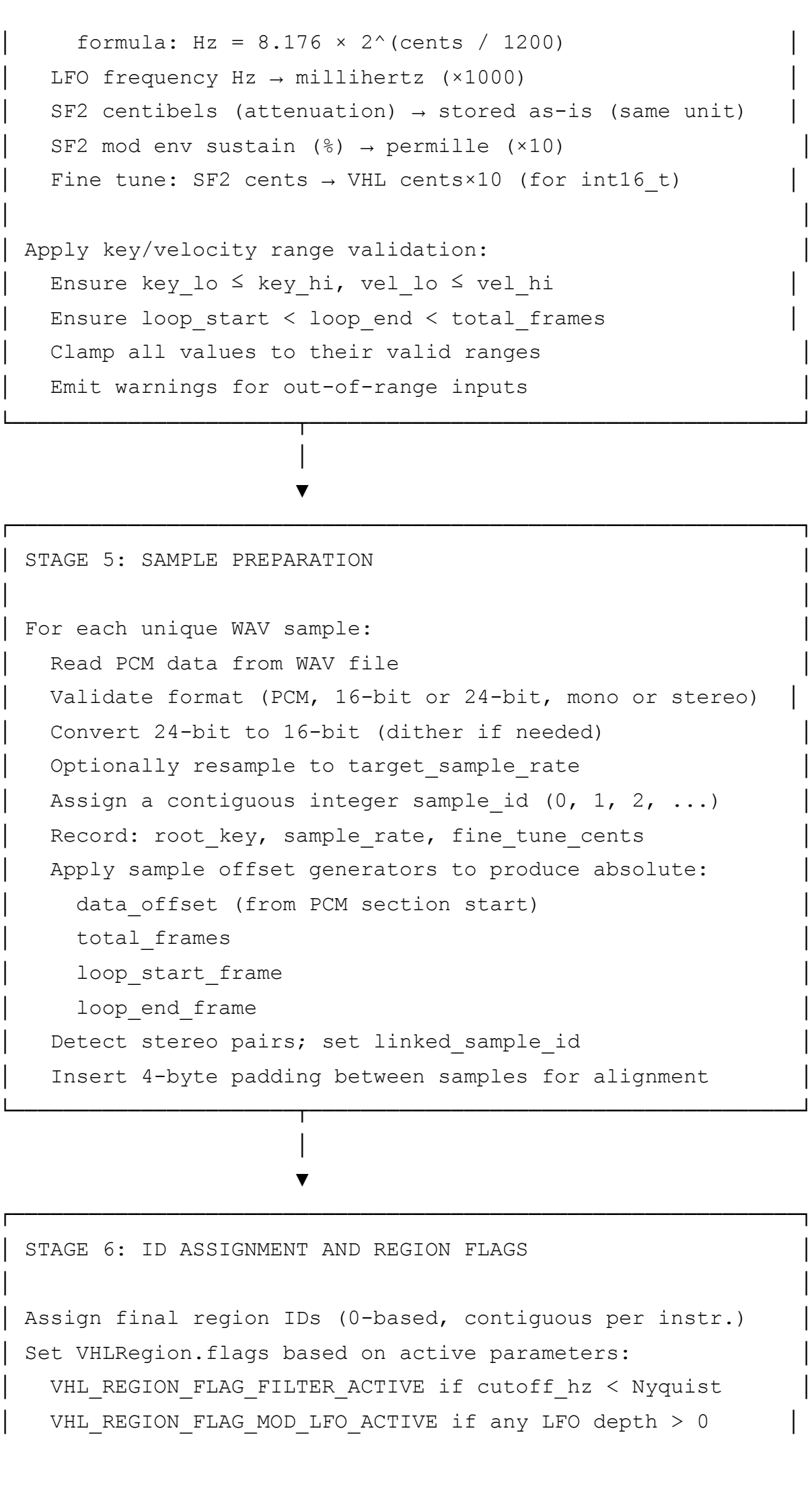
3. Builder Architecture

The Builder is an offline desktop tool (Python or C++). It has no real-time constraints. It may take seconds or minutes to complete. Its only requirement is correctness: the output VHL file must be valid, complete, and verified.

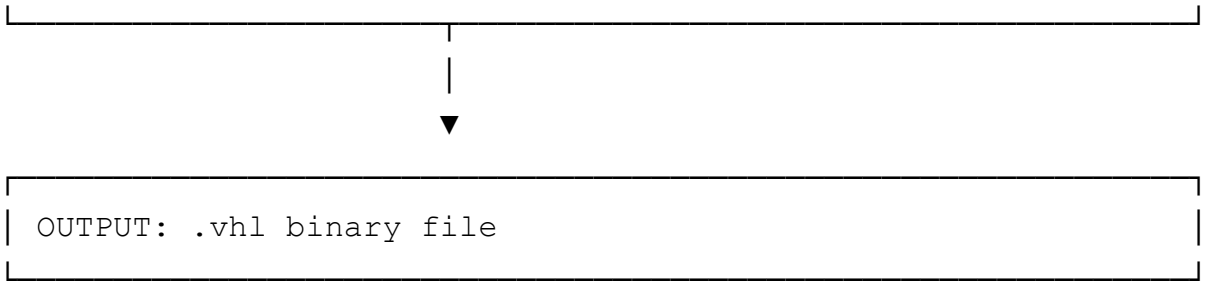
Pipeline Overview




Apply SFZ push-down rules:
global → group → region
For each region, compute its effective parameter value
for every opcode by following the inheritance chain.
For SF2 input (alternate path):
Resolve: sample default → inst global → inst zone
→ prst global → prst zone
For each inst zone × each matching prst zone:
Merge parameters with SF2 additive rules
Result: a flat list of fully self-contained regions.
No region has any reference to another region.
STAGE 3: MODULATOR PRE-APPLICATION
Apply SF2 default modulators that are note-independent:
CC7 default (100) → fold into attenuation_cb
CC10 default (64=center) → pan offset = 0, no change
CC11 default (127) → fold into attenuation_cb
CC91 default → fold into reverb_send
CC93 default → fold into chorus_send
Mark note-dependent modulators for runtime handling:
Velocity → attenuation: set VHL_REGION_FLAG_VEL_ATTEN
Pitch wheel: hardcoded ±200 cents in runtime
Result: every region has its final static parameter values.
STAGE 4: PARAMETER CONVERSION
Convert all parameters to VHL storage units:
SF2 timecents → milliseconds
formula: ms = 1000 × 2^(timecents / 1200)
SF2 absolute cents (filter freq, LFO freq) → Hz


formula: Hz = 8.176 × 2^(cents / 1200)
LFO frequency Hz → millihertz (×1000)
SF2 centibels (attenuation) → stored as-is (same unit)
SF2 mod env sustain (%) → permille (×10)
Fine tune: SF2 cents → VHL cents×10 (for int16_t)
Apply key/velocity range validation:
Ensure key_lo ≤ key_hi, vel_lo ≤ vel_hi
Ensure loop_start < loop_end < total_frames
Clamp all values to their valid ranges
Emit warnings for out-of-range inputs
STAGE 5: SAMPLE PREPARATION
For each unique WAV sample:
Read PCM data from WAV file
Validate format (PCM, 16-bit or 24-bit, mono or stereo)
Convert 24-bit to 16-bit (dither if needed)
Optionally resample to target_sample_rate
Assign a contiguous integer sample_id (0, 1, 2, ...)
Record: root_key, sample_rate, fine_tune_cents
Apply sample offset generators to produce absolute:
data_offset (from PCM section start)
total_frames
loop_start_frame
loop_end_frame
Detect stereo pairs; set linked_sample_id
Insert 4-byte padding between samples for alignment
STAGE 6: ID ASSIGNMENT AND REGION FLAGS
Assign final region IDs (0-based, contiguous per instr.)
Set VHLRegion.flags based on active parameters:
VHL_REGION_FLAG_FILTER_ACTIVE if cutoff_hz < Nyquist
VHL_REGION_FLAG_MOD_LFO_ACTIVE if any LFO depth > 0

```
|   VHL_REGION_FLAG_VIB_LFO_ACTIVE if vib depth > 0          |
|   VHL_REGION_FLAG_MOD_ENV_ACTIVE if mod env times > 0      |
|   VHL_REGION_FLAG_VEL_ATTEN if velocity modulation applies  |
|                                                             |
| These flags allow the runtime to skip DSP modules that      |
| have no effect (e.g., skip filter evaluation when all       |
| samples use the default open filter setting).               |
└─────────────────────────┬──────────────────────────────────┘
                          |
                          ▼
┌─────────────────────────────────────────────────────────────┐
| STAGE 7: LOOKUP TABLE GENERATION                            |
|                                                             |
| For each instrument:                                        |
|   Sort regions by key_lo                                    |
|   For each MIDI key (0–127):                                |
|     Find first region where key_lo ≤ key                    |
|     Count consecutive regions where key ≤ key_hi            |
|     Write VHLLookupEntry {region_start_idx, region_count}   |
|                                                             |
| This is an offline O(N×128) operation — trivially fast.     |
| At runtime, key lookup becomes O(1).                        |
└─────────────────────────┬──────────────────────────────────┘
                          |
                          ▼
┌─────────────────────────────────────────────────────────────┐
| STAGE 8: SERIALIZATION                                      |
|                                                             |
| Write sections in order:                                    |
|   Section 1: Instrument Table                               |
|   Section 2: Region Table                                   |
|   Section 3: Sample Directory                               |
|   Section 4: Lookup Tables                                  |
|   Section 5: PCM Sample Data                                |
|                                                             |
| Record byte offsets of each section start.                  |
| Write Section 0 (File Header) last, with all offsets,       |
| counts, and total file size filled in.                      |
|                                                             |
| Compute CRC32 of entire file (with crc32 field = 0).        |
| Patch crc32 field with computed value.                      |
```



Why Every Expensive Operation Belongs in the Builder

Operation	Cost on Desktop	Cost on ESP32	Decision
SFZ/SF2 parsing	~10 ms	~500 ms (no OS, slow storage)	Builder
Inheritance resolution	~1 ms	~50 ms (pointer chasing)	Builder
Modulator evaluation	~1 ms	~5 ms per Note-On	Builder
`pow(2, cents/1200)`	~1 µs	~20 µs each (no HW pow2)	Builder
Timecent → ms conversion	~1 µs	~20 µs each	Builder
WAV decoding	~100 ms	Not possible (no malloc)	Builder
Lookup table generation	~10 ms	Eliminated from runtime	Builder
CRC32 computation	~50 ms	Not performed (use stored)	Builder

---

4. Runtime Architecture

The ESP32 Runtime is a pure playback engine. It performs no interpretation, no parsing, and no offline computation. Every operation is bounded in time and deterministic in behavior.

Boot Loading Pipeline

```
Power On
    │
    ▼
Read 128 bytes from flash → VHLFileHeader
    │
    ├─ magic ≠ {'V','H','L','1'} → HALT (LED error code)
    ├─ version_major ≠ VHL_VERSION_MAJOR → HALT
    └─ CRC32 mismatch → HALT
    │
    ▼
```

```
Load into SRAM:
    VHLInstrumentEntry[num_instruments]      (~1.28 KB)
    VHLRegion[total_region_count]            (~8 KB)
    VHLSampleEntry[total_sample_count]       (~3.2 KB)
    VHLLookupHeader + VHLLookupEntry[]       (~16 KB max)
    │
    ▼
If PSRAM available:
    Copy PCM section → PSRAM (may take 1–3 seconds)
    Set pcm_base_ptr = PSRAM address
Else:
    Set pcm_base_ptr = flash XIP base + header.offset_pcm_data
    │
    ▼
Resolve VHLSampleEntry.data_ptr for each sample:
    sample[i].runtime_ptr = pcm_base_ptr + sample[i].data_offset
    │
    ▼
Initialize Voice Pool:
    memset(voice_pool, 0, sizeof(VHLVoice) × VHL_MAX_VOICES)
    │
    ▼
Initialize I²S DMA:
    sample_rate = header.target_sample_rate
    block_size  = header.dma_block_size
    Register DMA interrupt callback
    │
    ▼
System Ready — begin accepting MIDI
```

Note-On Processing Pipeline

This pipeline runs on the MIDI task thread (not in the DMA interrupt). Target: complete in under 20 µs.

```
MIDI Note-On received: {channel, key, velocity}
    │
    ▼
Resolve instrument:
    for each VHLInstrumentEntry:
        if entry.channel == channel AND entry.program matches:
            active_instrument = &instrument_table[i]
    │
```

```
    ▼
Find matching regions:
    lookup = lookup_table[instrument_id].entries[key]
    if lookup.region_start_idx == VHL_LOOKUP_NONE: return (no sound)
    │
    for i = 0 to lookup.region_count - 1:
        region = &region_table[lookup.region_start_idx + i]
        if vel_lo ≤ velocity ≤ vel_hi:
            → allocate voice for this region
    │
    ▼
For each matched region, allocate voice:
    │
    ├─ Exclusive class check:
    │    if region.exclusive_class != 0:
    │        scan active voices for same {exclusive_class, instrument_id,
channel}
    │        if found AND different key:
trigger_quick_release(found_voice)
    │
    ├─ Get voice from pool:
    │    scan voice_pool for first slot where active == 0
    │    if none: steal voice (prefer releasing → then oldest active)
    │
    ├─ Initialize voice state:
    │    voice.active = 1
    │    voice.releasing = 0
    │    voice.finished = 0
    │    voice.channel = channel
    │    voice.key = key
    │    voice.velocity = velocity
    │    voice.region_idx = matched_region_idx
    │    voice.sample_idx = region.sample_id
    │    voice.pos_integer = 0       (sample start)
    │    voice.pos_frac = 0
    │    voice.volume_coeff = 0      (ramp from silence)
    │    voice.exclusive_class = region.exclusive_class
    │
    ├─ Compute base pitch delta (once, at Note-On):
    │    root_key   = sample_entry.root_key
    │    fine_cents = region.fine_tune + sample_entry.fine_tune_cents×10
```

```
    │    voice.pitch_delta_cents =
    │        (key - root_key) × region.scale_tuning   // semitones × scale
    │        + region.coarse_tune × 100               // semitones → cents
    │        + fine_cents                              // fractional cents
    │        + sample_rate_compensation_cents          // rate mismatch
    │        (sample_rate_compensation is constant per sample,
precomputed)
    │
    ├─ Apply velocity attenuation (if VHL_REGION_FLAG_VEL_ATTEN):
    │    extra_atten = vel_curve[velocity]             // table lookup
    │    voice.applied_attenuation = region.attenuation_cb + extra_atten
    │
    └─ Initialize Envelope, LFO, Filter states:
         vol_env.phase = VHL_ENV_PHASE_DELAY (or ATTACK if delay_ms == 0)
         mod_env.phase = VHL_ENV_PHASE_DELAY
         lfo_mod.delay_done = 0, delta = 4×freq/sample_rate,
previous_point = 0
         lfo_vib.delay_done = 0
         filter.x1 = x2 = y1 = y2 = 0.0f
```

Note-Off Processing

```
MIDI Note-Off received: {channel, key}
    │
    for each active voice where voice.channel == channel AND voice.key ==
key:
        voice.releasing = 1
        vol_env.phase = VHL_ENV_PHASE_RELEASE
        if region.loop_mode == VHL_LOOP_SUSTAIN:
            voice.loop_active = 0  (exit loop, play to end)
```

DMA Interrupt Pipeline (Per Audio Block)

This is the critical real-time path. It must complete in under `block_size / sample_rate` seconds. For 512 samples at 44100 Hz: 11.6 ms maximum.

```
DMA interrupt fires (buffer empty)
    │
    ▼
Swap DMA buffers (pointer swap, O(1))
New buffer is now available for filling
```

```
    │
    ▼
Zero mixer accumulation buffers:
    memset(mix_L, 0, block_size × sizeof(float))
    memset(mix_R, 0, block_size × sizeof(float))
    │
    ▼
For each voice in voice_pool where voice.active == 1:
    │
    ├─ VOICE::PROCESS_BLOCK(voice, block_size)
    │
    │   For each CHUNK (256 samples) in this block:
    │
    │     ENVELOPE EVALUATION (per chunk):
    │       vol_env_val = advance_envelope(&vol_env, &region.vol_env, key)
    │       mod_env_val = advance_envelope(&mod_env, &region.mod_env, key)
    │       if vol_env.phase == VHL_ENV_PHASE_OFF: voice.finished = 1;
break
    │
    │     LFO EVALUATION (per chunk):
    │       mod_lfo_val = advance_lfo(&lfo_mod, &region.lfo_mod)
    │       vib_lfo_val = advance_lfo(&lfo_vib, &region.lfo_vib)
    │
    │     PITCH COMPUTATION (per chunk):
    │       current_cents = pitch_delta_cents
    │                     + pitch_wheel_cents[channel]
    │                     + region.mod_env_to_pitch × mod_env_val
    │                     + region.lfo_mod.to_pitch_cb × mod_lfo_val
    │                     + region.lfo_vib.to_pitch_cb × vib_lfo_val
    │       pitch_ratio_q16 = tuning_table_lookup(current_cents)
    │         (Q16.16 fixed-point: 65536 = ratio 1.0 = same pitch)
    │
    │     FILTER COEFFICIENT UPDATE (per chunk):
    │       if VHL_REGION_FLAG_FILTER_ACTIVE:
    │         effective_fc = region.filter.cutoff_hz
    │           × pow2_approx((mod_env_val × mod_env_to_filter
    │                         + mod_lfo_val × lfo_mod.to_filter_cb) /
1200.0)
    │         [coeffs[0], coeffs[1], coeffs[2]] = biquad_lp(effective_fc,
resonance)
    │
```

```
    |     VOLUME COEFFICIENT TARGET (per chunk):
    |       target_vol = s_conv_coef
    |                  × vol_env_val
    |                  × pow10_approx(0.05 × lfo_mod.to_volume_cb ×
mod_lfo_val)
    |       delta_vol = (target_vol - voice.volume_coeff) / VHL_CHUNK_SIZE
    |
    |     INNER SAMPLE LOOP (per sample, 256 iterations):
    |       voice.pos_frac += pitch_ratio_q16
    |       integer_advance = voice.pos_frac >> VHL_FRAC_BITS
    |       voice.pos_frac &= (1<<VHL_FRAC_BITS) - 1
    |       voice.pos_integer += integer_advance
    |
    |       Handle loop boundary:
    |         if pos_integer >= loop_end AND looping:
    |           pos_integer -= (loop_end - loop_start)  [wrap]
    |         elif pos_integer >= total_frames AND not looping:
    |           voice.finished = 1; break
    |
    |       Read 2 source samples for linear interpolation:
    |         s0 = pcm_data[pos_integer]
    |         s1 = pcm_data[pos_integer + 1]
    |       Interpolate:
    |         frac_f = voice.pos_frac * (1.0f / 65536.0f)
    |         sample = s0 + frac_f × (s1 - s0)
    |
    |       Apply filter (if active):
    |         y = coeffs[0]×(sample + x1+x1 + x2) + coeffs[1]×y1 +
coeffs[2]×y2
    |         x2=x1; x1=sample; y2=y1; y1=y
    |         sample = y
    |
    |       Apply volume:
    |         voice.volume_coeff += delta_vol
    |         output_sample = voice.volume_coeff × sample
    |
    |       Write to voice_buffer[sample_index]
    |
    ├─ MIXER ACCUMULATE:
    |   pan_L = pan_table_L[(region.pan + 500) × 255 / 1000]
    |   pan_R = pan_table_R[(region.pan + 500) × 255 / 1000]
```

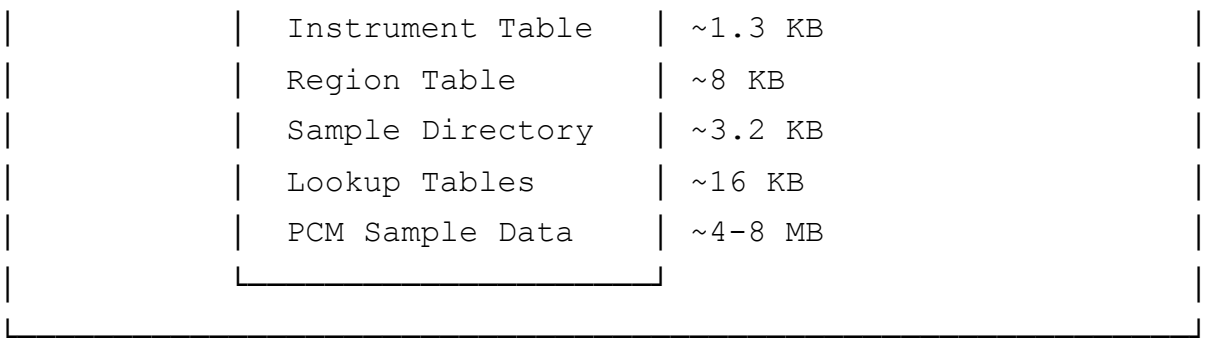
```
    │   for i = 0 to block_size - 1:
    │     mix_L[i] += voice_buffer[i] × pan_L
    │     mix_R[i] += voice_buffer[i] × pan_R
    │
    └─ If voice.finished: voice.active = 0
    │
    ▼
Hard clip output:
    for i = 0 to block_size - 1:
        mix_L[i] = clamp(mix_L[i], -1.0f, +1.0f)
        mix_R[i] = clamp(mix_R[i], -1.0f, +1.0f)
    │
    ▼
Convert float → int16 and write to DMA buffer:
    for i = 0 to block_size - 1:
        dma_buf[2i]   = (int16_t)(mix_L[i] × 32767.0f)
        dma_buf[2i+1] = (int16_t)(mix_R[i] × 32767.0f)
    │
    ▼
DMA peripheral reads new buffer → audio output
```

---

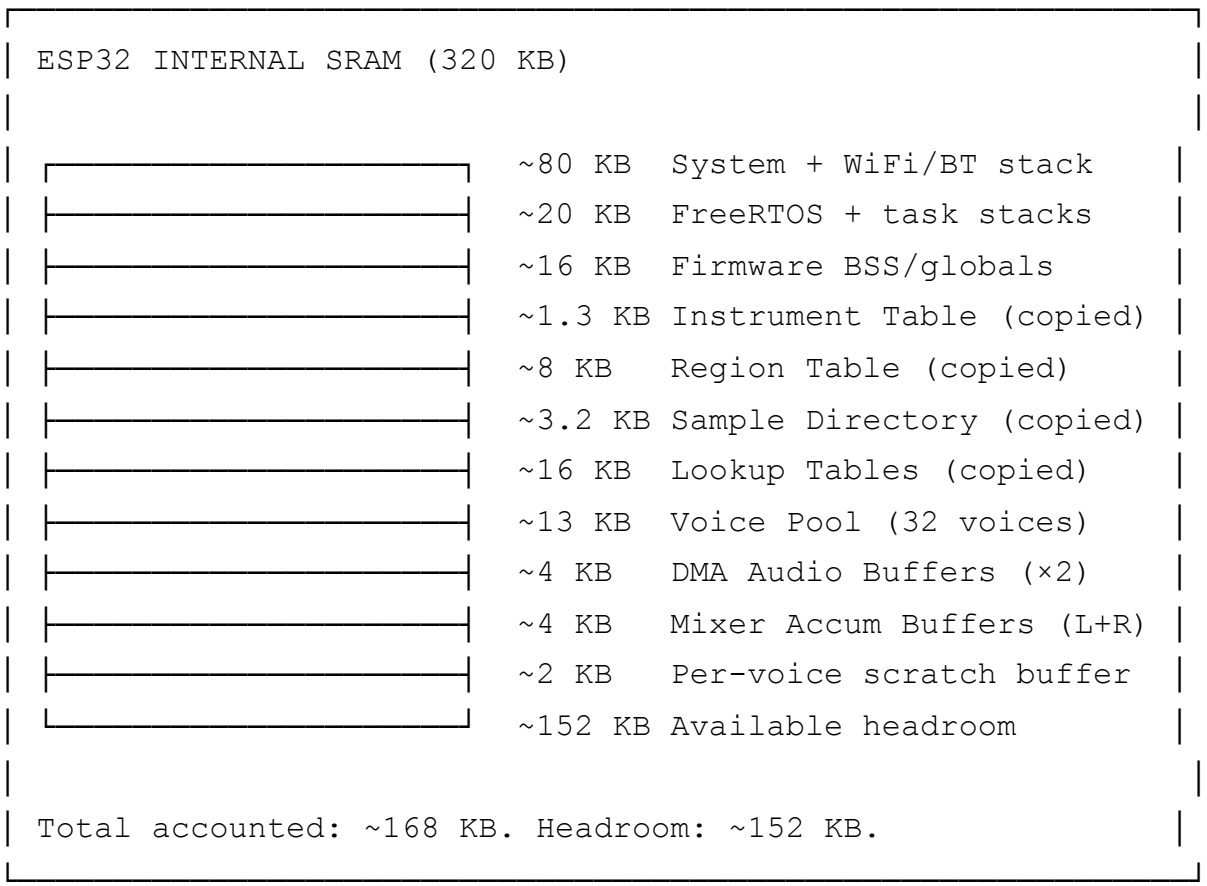
5. Memory Architecture

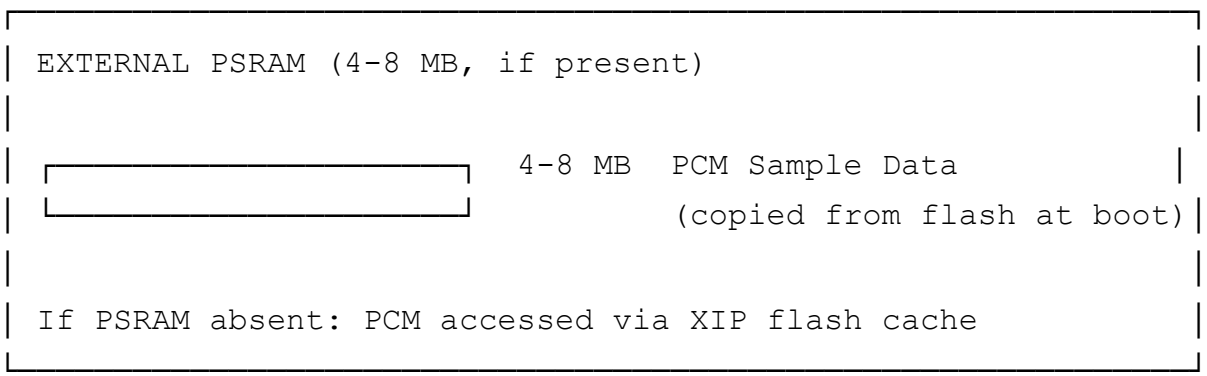
Flash Memory Map

```
┌──────────────────────────────────────────────────────────────┐
│ ESP32 FLASH (4-16 MB typical)                                │
│                                                              │
│ 0x000000 ┌──────────────────────┐                            │
│          │ Bootloader           │ ~64 KB (ESP-IDF reserved)  │
│ 0x010000 ├──────────────────────┤                            │
│          │ Partition Table      │ ~4 KB                      │
│ 0x011000 ├──────────────────────┤                            │
│          │ Firmware             │ ~1-2 MB                    │
│          │  (code + rodata)     │                            │
│          ├──────────────────────┤                            │
│          │ NVS (config)         │ ~16 KB                     │
│          ├──────────────────────┤                            │
│          │ VHL Package          │ Remaining flash            │
│          │  Header              │ 128 bytes                  │
```



RAM Memory Map





Voice Memory Budget

Each `VHLVoice` contains:

Component	Size
Identity fields	6 bytes
Region + sample index	4 bytes
Playback position	12 bytes
Pitch delta	4 bytes
Volume coefficient + gain	8 bytes
Envelope states (2 × 20 bytes)	40 bytes
LFO states (2 × 20 bytes)	40 bytes
Filter state	16 bytes
Per-chunk arrays (4 × 2 × 4 bytes)	32 bytes
Resampling scratch	16 bytes
Exclusive class + padding	4 bytes
Total	**~182 bytes**

Budgeted at 200 bytes per voice with alignment. 32 voices × 200 bytes = **6.4 KB**.

What Remains Permanently in Flash

- Firmware code (read via XIP instruction cache)
- VHL PCM Sample Data (read via XIP data cache, or copied to PSRAM at boot)
- VHL file itself (only the metadata sections are copied to SRAM; the PCM section stays in flash)

What Loads Into RAM

- VHL Instrument Table, Region Table, Sample Directory, Lookup Tables (copied at boot)
- Voice Pool (pre-allocated at boot, never freed)
- DMA audio buffers (ping-pong, permanent allocation)
- Mixer accumulation buffers (permanent allocation)

How Voices Reference Shared Samples Without Duplication

A `VHLVoice` contains `sample_idx` (uint16_t, 2 bytes). At voice initialization, the runtime resolves:

```
pcm_ptr = psram_base + sample_directory[sample_idx].data_offset
```

All voices that play the same sample share the exact same PCM data in PSRAM or flash. No copy is made per voice. 1000 simultaneous voices playing the same sample would use only 2 bytes per voice for the

reference, not 1 MB of PCM data per voice.

This is the central memory efficiency mechanism. The Region Table holds all per-zone parameters. The Sample Directory holds all per-sample metadata. The PCM section holds the raw audio. Voices hold only their per-note dynamic state.

---

6. Builder vs Runtime Responsibilities

Builder Responsibilities (Offline, Unlimited Resources)

Responsibility	Why It Belongs in the Builder
Parse SFZ / SF2 binary	Format parsing requires a real parser with error handling. Not suitable for embedded.
Read and decode WAV files	WAV decoding may require malloc, format negotiation, and decompression.
Resolve SFZ global→group→region inheritance	O(N×M) traversal per opcode. Fine for offline, unacceptable at Note-On.
Resolve SF2 preset→instrument→sample hierarchy	Two-stage pointer traversal. Completely eliminates at runtime.
Apply default modulators (CC7, CC10, CC11, etc.)	The modulator graph is complex. Results are static. Compute once, store forever.
Convert timecents → milliseconds	Requires `pow(2, x/1200)`. Called for every time parameter. Table or direct offline computation.
Convert absolute cents → Hz	Requires `pow(2, x/1200)` with different base. Same reason.
Convert 24-bit samples to 16-bit	Dithering requires random number generation and careful rounding. Not real-time.
Validate all parameter ranges	Error reporting and clamping with full diagnostic output. No display on ESP32.
Assign contiguous sample IDs	Requires full knowledge of all samples before assignment.
Compute absolute loop/start/end frame positions	Requires summing coarse + fine offset generators. Better done once offline.
Generate lookup tables (key × region mapping)	O(N×128) offline. Reduces Note-On scan from O(N) to O(1).
Compute CRC32 of output file	File must be complete before CRC can be computed.

Responsibility	Why It Belongs in the Builder
Pack binary file with correct offsets	All section sizes must be known before offsets can be written.

Runtime Responsibilities (Real-Time, Hard Deadline)

Responsibility	Why It Belongs in the Runtime
Boot validation (magic, version, CRC check)	Must happen at device startup. Trivial cost.
Copy metadata sections into SRAM	Must happen at boot. Direct memcpy from flash.
Preload PCM data to PSRAM	Must happen at boot. Direct DMA transfer.
Accept MIDI Note-On / Note-Off / CC events	Fundamental real-time input. Cannot be preprocessed.
Resolve bank+program → instrument	Per-event, O(N_instruments) scan = trivial.
Lookup key in acceleration table	Per-Note-On, O(1).
Scan velocity ranges for matching regions	Per-Note-On, O(M) where M ≤ 4. Deterministic.
Apply velocity→attenuation curve	Per-Note-On, O(1) table lookup.
Allocate voice from static pool	Per-Note-On, O(MAX_VOICES) scan = fast.
Initialize voice state	Per-Note-On, O(1) struct initialization.
Compute base pitch delta at Note-On	Per-Note-On, integer arithmetic only (after precomputation).
Handle exclusive class choke	Per-Note-On, O(MAX_VOICES) scan.
Handle Note-Off (trigger release phase)	Per-event, O(MAX_VOICES).
Advance envelope state machine	Per-chunk per-voice, 8-state machine.
Advance LFO oscillator	Per-chunk per-voice, triangle wave.
Compute pitch delta with LFO+env modulation	Per-chunk per-voice, integer additions.
Convert pitch delta to resampling ratio	Per-chunk, table lookup.
Resample PCM data (linear interpolation)	Per-sample per-voice, 2 multiplies + 1 add.
Manage loop boundaries	Per-sample, boundary detection + wrap.
Compute filter coefficients	Per-chunk, polynomial approximation.
Apply biquad filter	Per-sample, 5 multiplies + 4 adds.

Responsibility	Why It Belongs in the Runtime
Apply volume envelope with interpolation	Per-sample, 1 add + 1 multiply.
Accumulate voices into mixer	Per-sample, 2 multiplies (pan) + 2 adds.
Hard clip output	Per-sample, clamping operation.
Convert float → int16 for I^2S DMA	Per-sample, 1 multiply + cast.
Detect finished voices, return to pool	Per-block, O(MAX_VOICES).
Update pitch wheel per CC message	Per-event, store value. Applied at next block.

No responsibility overlaps. The Builder does what can be done once. The Runtime does only what must be done in real-time.

---

7. Future Expansion

The VHL format is designed for backward-compatible expansion. New features are added by:

1. Bumping `version_minor` (additive change — old runtimes ignore new fields)
2. Bumping `version_major` only for breaking changes (old runtimes refuse to load)
3. Using reserved fields in existing structs (each struct has `_reserved` fields)
4. Adding new sections (the header has 5 reserved `offset_reserved_N` fields)
5. Using `flags` bits (both file-level and region-level flags have reserved bits)

Stereo Samples

Current: `VHLSampleEntry.channels` field exists. `VHL_SAMPLE_FLAG_STEREO_RIGHT` flag exists. `linked_sample_id` field exists.

Expansion path: When stereo output is enabled (file flag `VHL_FILE_FLAG_STEREO_OUTPUT`), the runtime creates two voices per stereo region: one for the left channel sample and one for the right. No format change needed.

Round Robin

Current: `exclusive_class` provides a choke mechanism. Region flags have two reserved bits.

Expansion path: Add a `round_robin_group` byte to `VHLRegion` using one of the `_reserved` fields. The runtime maintains a per-group counter. Each Note-On selects the next region in the group (with matching key/vel) in rotation. Builder assigns consistent group IDs.

Release Samples

Current: `VHLRegion.flags` has reserved bits 6–7.

Expansion path: Bit 6 = `VHL_REGION_FLAG_ON_RELEASE`. At Note-Off, the runtime additionally searches for regions with this flag set on the same key, and creates a release-trigger voice. No other format change needed.

Drum Kits

Current: The existing region format supports one-sample-per-key instruments trivially — just define regions with single-key ranges and `loop_mode = VHL_LOOP_NONE`.

Expansion path: None required. Drum kits are natively supported by the existing format. The Builder simply creates regions where `key_lo == key_hi` for each drum instrument.

Layered Instruments

Current: Multiple regions with the same key/vel range already trigger multiple simultaneous voices (correct behavior per SF2 spec).

Expansion path: None required. Layers are natively supported.

Sample Streaming

Current: `VHLSampleEntry.data_offset` is a byte offset. The runtime resolves it to a pointer at boot.

Expansion path: Add a `VHL_SAMPLE_FLAG_STREAM` flag. When set, the runtime uses a prefetch ring buffer instead of a direct pointer. The streaming subsystem reads ahead by `2 × block_size` samples. No format change needed — the flag adds behavior, not structure.

Compression

Current: Samples are stored as raw 16-bit PCM.

Expansion path: Add `VHL_SAMPLE_FLAG_ADPCM` or `VHL_SAMPLE_FLAG_OPUS` flags. The runtime, if it supports decompression, decompresses into a local scratch buffer before resampling. The format structure does not change — only the PCM data interpretation changes. Compressed samples must set `total_frames` to the decoded frame count, not the compressed byte count.

Effects (Reverb, Chorus)

Current: `reverb_send` and `chorus_send` fields exist in `VHLRegion`. `VHL_FILE_FLAG_STEREO_OUTPUT` exists in the file header.

Expansion path: Add a new section (using one of the 5 reserved `offset_reserved_N` slots) containing effect configuration. The runtime's post-mixer stage reads this section and applies reverb/chorus. Minor version bump only. Old runtimes ignore the new section (their `offset_reserved_N` value is 0, so they skip it).

Multiple MIDI Channels

Current: `VHLInstrumentEntry.bank` and `VHLInstrumentEntry.program` exist. `VHLVoice.channel` exists.

Expansion path: Add a `VHLInstrumentEntry.channel` field using the existing `_reserved` byte. The runtime associates each instrument with a specific MIDI channel. Enables a General MIDI-style multi-timbral setup with no format change.

USB MIDI / Bluetooth MIDI

Current: The runtime architecture uses a `process_midi_event(channel, type, data1, data2)` interface.

Expansion path: The MIDI transport layer changes (add USB MIDI class driver or BLE-MIDI stack), but the event handler is identical. The VHL format is not involved.

ESP32-S3 Specific Optimizations

Current: The format and runtime design are hardware-agnostic.

Expansion path: The ESP32-S3 has more SRAM, faster PSRAM (OPI mode), and a dedicated DSP accelerator. On S3:

- Increase `VHL_MAX_VOICES` to 32 or 64
- Enable sinc resampling (8-tap) by default
- Enable reverb/chorus effects
  These are runtime configuration changes, not format changes.

Future Custom Instruments

Because the firmware is generic (it only understands VHL), adding a new instrument is a zero-firmware-change operation:

1. Record samples in any DAW
2. Import samples into Polyphone
3. Edit envelopes, loops, filters
4. Export SFZ from Polyphone
5. Run the VHL Builder on the SFZ
6. Copy the new `.vhl` file to the ESP32
7. Play

The ESP32 firmware never changes. The VHL format never changes (within major version). Only the data changes. This is the correct embedded instrument design philosophy.

End of VHL Architecture Document
*Together with `vhl_structs.h` and `parameter_mapping.csv`, this document constitutes the **VHL Specification v1.0** — the permanent foundation of the Virtual Harmonium Lab ecosystem.*